

AI认知诊断引擎

CDM驱动的知识状态诊断系统 · 代码开发记录

版本 1.0.0 | 2025年5月

⚠ 内容使用AI生成

技术栈: FastAPI + React 18 + TypeScript + PostgreSQL

架构: 5-Agent协作框架 + AgentBus消息总线

算法: DINA模型 · EM参数估计 · 根因追溯 · 反事实推理 · 遗忘曲线

📖 目录

第一章 项目概览

第二章 技术架构

第三章 后端开发记录

第四章 前端开发记录

第五章 API接口清单

第六章 数据库设计

第七章 核心算法实现

第八章 Agent系统设计

第九章 测试与质量

第一章 项目概览

1.1 项目定位

AI认知诊断引擎是一个面向K-12教育的智能教学辅助平台，以认知诊断理论（CDM）为算法核心，帮助教师从答题数据中量化推断学生的知识状态，实现**从经验驱动到数据驱动**的教学决策转型。

1.2 代码规模



1.3 开发阶段

阶段	内容	状态
Phase 1	基础框架搭建：FastAPI项目结构、数据库ORM、DINA模型原型	✅ 完成
Phase 2	CDM核心算法：EM参数估计、蒙特卡洛EM、拟合优度（AIC/BIC）	✅ 完成
Phase 3	Agent框架：5个Agent + AgentBus消息总线 + 事件系统	✅ 完成
Phase 4	前端页面：13个功能页面 + 路由 + 状态管理 + API对接	✅ 完成
Phase 5	组件库：21个可复用组件（表格/图表/树控件/时间线等）	✅ 完成
Phase 6	测试：后端54用例 + 前端24用例 + TypeScript类型检查	✅ 完成
Phase 7	Docker部署：三服务容器编排 + 健康检查 + 数据初始化	✅ 完成

1.4 技术选型理由

技术	选择理由
FastAPI（后端框架）	原生异步支持、自动OpenAPI文档生成、Pydantic数据校验，适合计算密集型诊断API
React 18 + TypeScript	成熟的UI生态（Ant Design 5）、严格类型系统保障大规模组件协作
PostgreSQL + SQLite	生产环境使用PostgreSQL保证并发和数据完整性；开发环境可用SQLite零配置
NumPy + SciPy	CDM的EM算法依赖高效矩阵运算和优化函数，Python科学计算事实标准
@ant-design/charts	基于ECharts的React封装，提供趋势图、热力图、折线图教育可视化所需图表
Zustand	比Redux更轻量的状态管理，适合中等规模应用，API简洁
Docker Compose	三服务一键部署，降低环境配置门槛

第二章 技术架构

2.1 整体架构

用户层

浏览器 (Chrome / Edge / Safari) → React 18 SPA (Ant Design 5 UI)

代理层 (Nginx in Docker)

静态文件托管 + /api/* 反向代理到后端:8003

应用层 (FastAPI Backend :8003)

Controller (API Routes) → Service Layer (DiagnosisService · KnowledgeService · TracingService · TeachingService · EvolutionService) → Agent Layer (DiagnosisAgent · KnowledgeAgent · TracingAgent · TeachingAgent · EvolutionAgent) → CDM Engine (DINA | DINO | BKT)

AgentBus 消息总线

DiagnosisAgent → TracingAgent + TeachingAgent
KnowledgeAgent → DiagnosisAgent
EvolutionAgent → 全Agent (在线学习信号)

数据层

PostgreSQL 16 · SQLite (开发) · SQLAlchemy ORM · Alembic 迁移

2.2 后端目录结构

```
backend/  
├── app/
```

```

|   |—— main.py           # FastAPI 应用入口 + lifespan
|   |—— config.py         # 配置管理 (pydantic-settings)
|   |—— api/v1/           # REST API 路由层
|   |   |—— agent.py      # Agent监控 API
|   |   |—— class_api.py  # 班级管理 API
|   |   |—— courseware.py # 课件管理 API
|   |   |—— curriculum.py # 课标数据 API
|   |   |—— diagnosis.py  # 认知诊断 API (核心)
|   |   |—— homework.py  # 作业管理 API
|   |   |—— knowledge.py  # 知识图谱 API
|   |   |—— project.py    # 项目管理 API
|   |—— models/
|   |   |—— db_models.py  # SQLAlchemy ORM 模型 (13表)
|   |   |—— schemas.py   # Pydantic 请求/响应模型
|   |—— services/
|   |   |—— agent_bus.py  # AgentBus 消息总线
|   |   |—— agent_diagnosis.py # 诊断Agent
|   |   |—— agent_knowledge.py # 知识Agent
|   |   |—— agent_tracing.py # 追踪Agent
|   |   |—— agent_teaching.py # 教学Agent
|   |   |—— agent_evolution.py # 演化Agent
|   |   |—— dina_model.py  # DINA模型 + EM算法
|   |   |—— diagnosis_service.py # 诊断业务流程
|   |   |—— knowledge_service.py # 知识图谱服务
|   |   |—— project_service.py # 项目CRUD服务
|—— data/curriculum/      # 课标JSON数据文件
|—— tests/                # 测试用例
|—— init_data.py          # 数据库初始化脚本
|—— Dockerfile            # 后端Docker镜像
|—— requirements.txt      # Python依赖清单

```

2.3 前端目录结构

```

frontend/
|—— src/
|   |—— App.tsx           # 根组件 + 路由定义
|   |—— types/index.ts    # TypeScript 类型定义
|   |—— services/         # API 服务层
|   |   |—— api.ts        # Axios 实例 + 拦截器
|   |   |—— diagnosis.ts  # 诊断API
|   |   |—— homework.ts  # 作业API
|   |   |—— project.ts    # 项目API
|   |   |—— agent.ts      # Agent API
|   |—— stores/           # Zustand 状态管理
|   |   |—— useProjectStore.ts
|   |   |—— useNotificationStore.ts
|   |—— pages/            # 13个功能页面
|   |   |—— Dashboard/    # 系统总览

```

```
| | |—— ProjectManage/ # 项目管理
| | |—— CoursewareManage/ # 课件管理
| | |—— KnowledgeOverview/# 知识图谱
| | |—— KnowledgeDetail/ # 知识点详情
| | |—— HomeworkManage/ # 作业管理
| | |—— QMatrixEditor/ # Q矩阵编辑
| | |—— DiagnosisOverview/ # 诊断总览
| | |—— StudentDiagnosis/ # 学生诊断
| | |—— DiagnosisDetail/ # 诊断分析
| | |—— TeachingDecision/ # 教学决策
| | |—— AgentMonitor/ # Agent监控
| | |—— SystemSettings/ # 系统设置
| |—— components/ # 21个可复用组件
| | |—— AgentMessageFlow/ # Agent消息流
| | |—— AgentStatusCard/ # Agent状态卡片
| | |—— CDMPParams/ # CDM参数展示
| | |—— CounterfactualPanel/ # 反事实预测
| | |—— CoursewareModeTag/ # 课件模式标签
| | |—— DataTable/ # 通用数据表格
| | |—— DiagnosisReport/ # 诊断报告
| | |—— ExamMethodTag/ # 考法标签
| | |—— ExamPointList/ # 考点列表
| | |—— ForgettingCurve/ # 遗忘曲线
| | |—— KnowledgeGraph/ # 知识图谱 (G6)
| | |—— KnowledgeHierarchy/ # 知识层级树
| | |—— LearningPathTimeline/ # 学习路径时间线
| | |—— MasteryHeatmap/ # 掌握率热力图
| | |—— MasteryTrajectory/ # 掌握轨迹图
| | |—— ParseStatusTag/ # 解析状态标签
| | |—— ProjectSelector/ # 项目选择器
| | |—— QMatrixEditor/ # Q矩阵编辑组件
| | |—— RootCauseTree/ # 根因追溯树
| | |—— StatCard/ # 统计卡片
| | |—— StudentCard/ # 学生信息卡片
| | |—— TrendChart/ # 趋势图
| |—— styles/tokens.css # 设计令牌 (主题色)
| |—— test/ # 测试文件
| | |—— setup.ts # 测试环境配置
| | |—— DataTable.test.tsx
| | |—— CoreComponents.test.tsx
| | |—— ChartComponents.test.tsx
| | |—— PComponents.test.tsx
| |—— main.tsx # 应用入口
|—— Dockerfile # 前端Docker镜像
|—— nginx.conf # Nginx配置
|—— package.json # npm依赖
```

第三章 后端开发记录

3.1 FastAPI 应用入口

文件: app/main.py

应用采用 **lifespan 上下文管理器** 模式管理启动/关闭生命周期:

- **启动阶段**: 初始化所有5个Agent → 构建AgentBus → 注册消息处理器 → 预加载知识图谱
- **运行阶段**: 提供REST API、WebSocket (预留)、后台任务处理
- **关闭阶段**: 优雅关闭Agent任务、释放数据库连接

应用注册了7个路由模块, 覆盖项目、课件、知识图谱、作业、诊断、Agent监控、班级管理
等全部业务域。

3.2 DINA模型实现

文件: app/services/dina_model.py

组件	说明
响应概率计算	$P(R_{ij}=1 \alpha_i) = (1-s_j)^{\eta_{ij}} \times g_j^{1-\eta_{ij}}$ 其中 $\eta_{ij} = \prod_k \alpha_{ik}^{q_{jk}}$ 为理想响应
EM算法-E步	计算每个学生属于每个可能知识状态的后验概率 $P(\alpha R)$, 基于当前模型参数
EM算法-M步	最大化期望对数似然, 更新 slip/guess 参数和状态分布先验
蒙特卡洛模式	$K > 15$ 时状态空间 $2^K > 32768$, 改用随机采样逼近E步积分
收敛判断	连续两次迭代的相对参数变化 $< 10^{-6}$ 时视为收敛

3.3 配置管理

文件: app/config.py

使用 pydantic-settings 实现类型安全的配置管理, 支持从环境变量和 .env 文件读取配置。
主要配置项包括数据库连接串、CORS跨域白名单、服务端口等。

第四章 前端开发记录

4.1 主题色彩系统

文件: src/styles/tokens.css

前端定义了完整的设计令牌 (Design Tokens) 体系, 统一管理全局视觉样式:

变量	值	用途
--color-primary	#1677ff	主色调 (按钮、链接、强调)
--color-cdm-high	#52c41a	CDM高掌握率 (绿色)
--color-cdm-medium	#faad14	CDM中等掌握率 (橙色)
--color-cdm-low	#ff4d4f	CDM低掌握率 / 根因 (红色)
--color-counterfactual	#722ed1	反事实推理 / Agent标识 (紫色)
--color-bg-layout	#f5f5f5	页面背景

4.2 状态管理方案

前端采用 **Zustand** 轻量状态管理, 共2个 store:

Store	文件	管理状态
useProjectStore	stores/useProjectStore.ts	当前项目ID/名称、项目列表、项目切换
useNotificationStore	stores/useNotificationStore.ts	全局通知消息、通知计数、通知设置

4.3 API服务层设计

文件: src/services/api.ts

基于Axios封装统一的HTTP客户端, 包含:

- Base URL 配置 (从 VITE_API_BASE_URL 环境变量读取)
- 请求/响应拦截器 (错误处理、Token注入预留)
- 诊断专用API模块 (diagnosis.ts)

- 作业管理API模块 (homework.ts)
- 项目管理API模块 (project.ts)
- Agent监控API模块 (agent.ts)

第五章 API接口清单

5.1 项目管理接口

路由前缀: /api/v1/projects

方法	路径	说明
GET	/api/v1/projects	获取项目列表
POST	/api/v1/projects	创建项目
GET	/api/v1/projects/{id}	获取项目详情
PUT	/api/v1/projects/{id}	更新项目
DELETE	/api/v1/projects/{id}	删除项目

5.2 课件管理接口

路由前缀: /api/v1/coursewares

方法	路径	说明
GET	/api/v1/coursewares	课件列表（支持按项目/状态筛选）
POST	/api/v1/coursewares/upload	上传PPT课件（multipart）
GET	/api/v1/coursewares/{id}	课件详情（含幻灯片解析结果）
POST	/api/v1/coursewares/{id}/parse	触发课件解析

5.3 知识图谱接口

路由前缀: /api/v1/knowledge

方法	路径	说明
GET	/api/v1/knowledge/tree	获取知识层级树（支持学科/层级筛选）
GET	/api/v1/knowledge/graph	获取知识图谱数据（节点+边）

GET	/api/v1/knowledge/{kp_id}	知识点详情（含前驱/后继链）
GET	/api/v1/knowledge/exam-points	考点列表（含考频/难度）

5.4 作业管理接口

路由前缀: /api/v1/homeworks

方法	路径	说明
GET	/api/v1/homeworks	作业列表
POST	/api/v1/homeworks	创建作业
GET	/api/v1/homeworks/{id}	作业详情（含题目列表）
POST	/api/v1/homeworks/{id}/items	添加题目
POST	/api/v1/homeworks/{id}/answers/import	导入答题数据（Excel/CSV）
GET	/api/v1/homeworks/{id}/qmatrix	获取Q矩阵
PUT	/api/v1/homeworks/{id}/qmatrix	更新Q矩阵

5.5 认知诊断接口（核心）

路由前缀: /api/v1/diagnosis

方法	路径	说明
POST	/api/v1/diagnosis/estimate	执行CDM参数估计（DINA/DINO）
GET	/api/v1/diagnosis/sessions	诊断会话列表
GET	/api/v1/diagnosis/sessions/{id}	诊断会话详情（slip/guess/alpha）
GET	/api/v1/diagnosis/sessions/{id}/overview	班级诊断概览（平均掌握率、薄弱排行）
GET	/api/v1/diagnosis/sessions/{id}/student/{sid}	单个学生诊断结果
GET	/api/v1/diagnosis/sessions/{id}/student/{sid}/root-cause	单个学生根因追溯

GET	/api/v1/diagnosis/sessions/{id}/student/{sid}/counterfactual	反事实预测
GET	/api/v1/diagnosis/sessions/{id}/student/{sid}/learning-path	个性化学习路径
GET	/api/v1/diagnosis/sessions/{id}/mastery-heatmap	班级掌握率热力图数据
GET	/api/v1/diagnosis/teaching/suggestions	教学建议（补课优先级/学生分组/干预方案）

5.6 Agent监控接口

路由前缀: /api/v1/agent

方法	路径	说明
GET	/api/v1/agent/status	所有Agent状态列表
GET	/api/v1/agent/status/{name}	指定Agent状态详情
GET	/api/v1/agent/events	Agent事件日志（分页）
GET	/api/v1/agent/messages	Agent消息流记录

5.7 其他接口

方法	路径	说明
GET	/api/v1/classes	班级列表
GET	/api/v1/students	学生列表（按班级筛选）
GET	/api/v1/curriculum	课标数据列表（按学科筛选）
POST	/api/v1/curriculum/import	导入课标数据
GET	/health	服务健康检查

第六章 数据库设计

6.1 ER关系简述

Project —1:N—> Courseware —1:N—> Slide —N:M—> KnowledgePoint
Project —1:N—> Homework —1:N—> HomeworkItem —N:M—>
KnowledgePoint (via Q-Matrix)
HomeworkItem —1:N—> AnswerRecord
Project —1:N—> CDMSession —1:N—> CDMResult
CDMResult —1:N—> RootCause —N:1—> KnowledgePoint
CDMResult —1:1—> LearningPath
KnowledgePoint —N:M—> KnowledgePoint (prerequisites)

6.2 核心表详细说明

knowledge_points (知识点表)

列名	类型	说明
id	UUID	主键
code	VARCHAR(50)	编码 (如 GEO_CLIMATE_001)
name	VARCHAR(200)	名称
level	INTEGER	层级 0-5 (学科→考法)
parent_id	UUID FK	父节点 (自引用)
subject	VARCHAR(50)	所属学科
description	TEXT	描述
cognitive_level	VARCHAR(50)	认知层次 (识记/理解/应用/综合)

cdm_results (诊断结果表)

列名	类型	说明
id	UUID	主键
session_id	UUID FK	诊断会话ID
student_id	VARCHAR(100)	学生ID
kp_id	UUID FK	知识点ID
P_mastery	FLOAT	后验掌握概率 (0~1)

cdm_sessions (诊断会话表)

列名	类型	说明
id	UUID	主键
homework_id	UUID FK	关联作业
model_type	VARCHAR(20)	DINA DINO
status	VARCHAR(20)	running completed failed
iterations	INTEGER	EM迭代次数
converged	BOOLEAN	是否收敛
AIC	FLOAT	AIC拟合指标
BIC	FLOAT	BIC拟合指标

q_matrix (Q矩阵表)

列名	类型	说明
id	UUID	主键
item_id	UUID FK	题目ID
kp_id	UUID FK	考察的知识点ID
value	INTEGER	0: 不考察; 1: 考察
confirmed	BOOLEAN	是否经教师确认

第七章 核心算法实现

7.1 DINA模型响应概率

DINA模型定义学生 i 答对题目 j 的概率为：

$$P(R_{ij} = 1 \mid \alpha_i) = (1 - s_j)^{\eta_{ij}} \cdot g_j^{1 - \eta_{ij}}$$

其中 $\eta_{ij} = \prod_{k=1}^K \alpha_{ik}^{q_{jk}}$ ，当学生掌握了题目考核的所有知识点时 $\eta=1$ （反映为“理想掌握”），否则 $\eta=0$ 。

7.2 EM参数估计算法

伪代码：

```
输入：Q矩阵(M×K)，答题矩阵R(S×M)，最大迭代I_max
输出：slip向量(s1,...,s_M)，guess向量(g1,...,g_M)，alpha后验

初始化：s_j=0.2, g_j=0.2, 状态先验均匀分布

for iter in 1..I_max:
    # E步：计算后验
    for each student i:
        for each 可能状态  $\alpha \in \{0,1\}^K$ :
            计算  $P(\alpha \mid R_i) = P(R_i \mid \alpha) \times P(\alpha) / \sum P(R_i \mid \alpha') \times P(\alpha')$ 

    # 基于后验计算期望充分统计量
    for each question j:
         $n_{j00} = \sum_i \sum_{\alpha: \eta=0} P(\alpha \mid R_i) \times (1-R_{ij})$  #  $\eta=0$ 且答错
         $n_{j01} = \sum_i \sum_{\alpha: \eta=0} P(\alpha \mid R_i) \times R_{ij}$  #  $\eta=0$ 且答对

    # M步：更新参数
     $g_j = n_{j01} / (n_{j00} + n_{j01})$ 
     $s_j = 1 - n_{j11} / (n_{j10} + n_{j11})$ 

    if max|param_new - param_old| < 1e-6: break
```

7.3 蒙特卡洛EM ($n_{kp} > 15$)

当 $K>15$ 时， 2^K 超过32768，枚举所有状态不可行。降级策略：

- 从历史 α 后验分布中采样 $M=500$ 个代表性状态
- 用采样状态近似E步积分： $P(\alpha | R) \approx P(R | \alpha) / \sum_{\alpha' \in \text{Sample}} P(R | \alpha')$
- 降低迭代次数上限至5轮，平衡精度和性能

7.4 根因追溯算法

输入：学生 i 的 α 向量、知识图谱 G 、薄弱阈值 θ_w （默认0.4）

```

对于每个  $P_{\text{mastery}}(ik) < \theta_w$  的薄弱知识点  $k$ :
    candidates = []
    沿 $G$ 中的前驱链追溯:
        对每个前驱节点  $p$ :
            score_p = depth(p)  $\times$  (1 -  $P_{\text{mastery}}(ip)$ )
            candidates.append({node: p, score: score_p})

    root_cause = argmax(candidates.score) # 最深 $\times$ 最薄弱
    构建从 root_cause 到  $k$  的追溯路径树

返回：每个薄弱知识点的根因节点 + 路径

```

7.5 反事实推理

基于CDM参数化模型：假设学生 i 的知识状态从 α_i 变为 α'_i （补上某个知识点），重新计算所有知识点的 $P(\text{掌握})$ ，量化“如果补上 X ， Y 的掌握率将从 $A\%$ 变为 $B\%$ ”。

7.6 遗忘曲线

基于Ebbinghaus指数衰减模型： $M(t) = M_0 \cdot e^{-\lambda t}$

- M_0 ：初始掌握率（诊断时）
- λ ：衰减率参数（由历史数据拟合）
- 半衰期 $t_{1/2} = \ln(2) / \lambda$ （推荐复习时间）

第八章 Agent系统设计

8.1 AgentBus消息总线

文件: app/services/agent_bus.py

AgentBus是Agent间通信的核心基础设施，采用发布-订阅模式：

- 注册机制**：每个Agent在初始化时向Bus注册，声明订阅的消息类型
- 消息路由**：Agent发送消息时指定类型，Bus根据注册表路由给订阅方
- 事件日志**：所有消息均持久化到 agent_events 表，用于监控和审计

8.2 消息流设计

消息类型	发送方	接收方	触发条件
graph_updated	KnowledgeAgent	DiagnosisAgent	课件解析完成后知识图谱更新
diagnosis_completed	DiagnosisAgent	TracingAgent, TeachingAgent	CDM估计完成
trace_updated	TracingAgent	TeachingAgent	BKT追踪更新
teaching_plan_generated	TeachingAgent	DiagnosisAgent	教学方案生成
new_data_arrived	外部触发	EvolutionAgent	新答题数据到达
model_evolved	EvolutionAgent	全Agent	在线学习参数更新

8.3 Agent生命周期

- 初始化**：Agent实例化 → 加载配置 → 订阅消息 → 设置事件处理器
- 运行**：监听消息 → 处理业务逻辑 → 发送结果消息
- 空闲**：无任务时进入等待状态
- 异常**：处理失败时记录错误日志，触发告警通知
- 关闭**：收到关闭信号 → 完成当前任务 → 保存状态 → 取消订阅

第九章 测试与质量

9.1 后端测试

测试文件	测试内容
tests/test_dina.py	DINA模型：响应概率计算、EM收敛性、slip/guess参数恢复、蒙特卡洛模式、边界条件（全正确/全错误数据）
tests/test_api_core.py	核心API：项目管理CRUD、课件上传、作业创建、诊断流程端到端
tests/test_agent.py	Agent系统：注册/取消注册、消息发送接收、事件日志
tests/test_knowledge.py	知识图谱：层级树查询、前驱链追溯、课标导入

总计：54个测试用例通过，7个慢速测试（蒙特卡洛采样、大数据集）在CI中跳过。

9.2 前端测试

测试文件	用例数	内容
DataTable.test.tsx	3	渲染、搜索框显示/隐藏
CoreComponents.test.tsx	8	RootCauseTree（数据/空数据）、LearningPathTimeline、KnowledgeHierarchy、AgentStatusCard（正常/加载中）
ChartComponents.test.tsx	4	TrendChart、MasteryHeatmap、ForgettingCurve、MasteryTrajectory（均mock @ant-design/charts）
P1Components.test.tsx	9	ProjectSelector、StudentCard、ExamPointList、ExamMethodTag（4种类型）、DiagnosisReport（数据/空数据）、AgentMessageFlow（数据/空数据）

总计：4个文件、24个测试用例、100%通过率。

9.3 类型安全

前端使用 TypeScript 严格模式，通过 `npx tsc --noEmit` 确保零编译错误。所有组件 Props、API 响应、Store 状态均有完整类型定义。

第十章 部署架构

10.1 Docker Compose 编排



10.2 健康检查策略

服务	检查方式	间隔	超时	重试
postgres	pg_isready -U postgres	10s	5s	5
backend	wget --spider /health	10s	5s	5
frontend	— (依赖backend健康)	—	—	—

10.3 启动依赖链

postgres (健康) → backend (健康) → frontend。通过 Docker Compose 的 `depends_on: condition: service_healthy` 保证启动顺序。

AI认知诊断引擎 · 代码开发记录 v1.0.0

⚠ 本文档内容使用AI生成 | © 2025 AI认知诊断引擎